

# Day 3: Agentic Systems

---

Agentic AI Intensive Training

5-Day Program

# Today's Agenda

---

## Morning: Agent Foundations

- What are agents?
- Core architectures
- ReAct pattern

## Afternoon: Tool Use & Multi-Agent

- Function calling
- Tool integration
- Agent collaboration

# What is an Agent?

---

## Definition

- Autonomous system that perceives and acts
- Uses LLM for reasoning/decision-making
- Interacts with tools/environment
- Pursues goals over multiple steps

## Key Characteristics

- Goal-directed
- Adaptive

# Agent vs Standard LLM

---

## Standard LLM

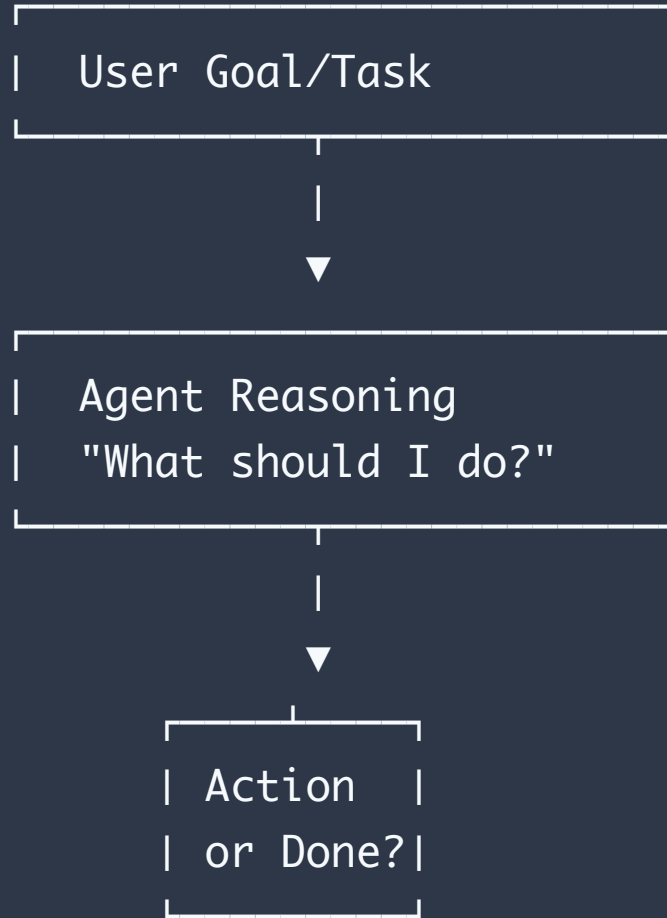
- Single request/response
- No environment interaction
- Stateless (without implementation)
- Direct output

## Agent

- Multi-turn reasoning loop
- Tool/API calls

# Simple Agent Loop

---



# Core Agent Architectures

---

## 1. ReAct (Reason + Act)

- Interleave reasoning and actions
- Explicit thought traces

## 2. Plan-and-Execute

- Create plan first
- Execute steps sequentially

## 3. Reflexion

# ReAct Pattern

---

Reason → Act → Observe → Repeat

Thought: I need to find population of Tokyo

Action: `search("Tokyo population 2024")`

Observation: Tokyo has 14 million people

Thought: Now I need Paris population

Action: `search("Paris population 2024")`

Observation: Paris has 2.1 million people

Thought: I can now compare

Answer: Tokyo is ~6.7x larger than Paris

# Python: Simple ReAct Agent

---

```
def react_agent(question, tools, max_steps=5):
    context = f"Question: {question}\n"

    for step in range(max_steps):
        # Reasoning
        thought = llm(f"{context}\nThought:")
        context += f"\nThought: {thought}"

        # Decide action or finish
        if "Answer:" in thought:
            return extract_answer(thought)

        # Execute action
        action = parse_action(thought)
        result = execute_tool(action, tools)
```

# TypeScript: Simple ReAct Agent

---

```
async function reactAgent(  
  question: string,  
  tools: Map<string, Function>,  
  maxSteps = 5  
) : Promise<string> {  
  let context = `Question: ${question}\n`;  
  
  for (let step = 0; step < maxSteps; step++) {  
    const thought = await llm(`${context}\nThought:`);  
    context += `\nThought: ${thought}`;  
  
    if (thought.includes("Answer:")) {  
      return extractAnswer(thought);  
    }  
  }  
}
```

# Tools and Functions

---

## What are Tools?

- Functions agents can call
- Access external data/systems
- Perform actions (API calls, calculations)

## Common Tool Categories

- Search (web, docs, databases)
- Computation (calculator, code exec)
- Data retrieval (APIs, databases)

# Tool Definition Schema

---

```
tools = [  
  {  
    "name": "web_search",  
    "description": "Search the web for information",  
    "parameters": {  
      "type": "object",  
      "properties": {  
        "query": {  
          "type": "string",  
          "description": "Search query"  
        }  
      },  
      "required": ["query"]  
    }  
  }  
]
```

# OpenAI Function Calling

---

```
from openai import OpenAI

client = OpenAI()

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string"},
                "unit": {"type": "string", "enum": ["C", "F"]}
            }
        }
    }
}]
```

# Python: Function Calling Flow

---

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    messages=[{"role": "user", "content": "Weather in NYC?"}],  
    tools=tools  
)  
  
# Check if function called  
tool_call = response.choices[0].message.tool_calls[0]  
if tool_call:  
    function_name = tool_call.function.name  
    args = json.loads(tool_call.function.arguments)  
  
    # Execute function  
    result = get_weather(**args)
```

# TypeScript: Function Calling

---

```
const response = await client.chat.completions.create({
  model: "gpt-4o",
  messages: [{ role: "user", content: "Weather in NYC?" }],
  tools: tools
});

const toolCall = response.choices[0].message.tool_calls?.[0];
if (toolCall) {
  const functionName = toolCall.function.name;
  const args = JSON.parse(toolCall.function.arguments);

  // Execute function
  const result = await getWeather(args);

  // Send result back
```

# Building a Calculator Tool

---

```
def calculator_tool(operation: str, x: float, y: float) -> float:
    """Perform mathematical operations"""
    ops = {
        "add": lambda a, b: a + b,
        "subtract": lambda a, b: a - b,
        "multiply": lambda a, b: a * b,
        "divide": lambda a, b: a / b if b != 0 else None
    }
    return ops.get(operation, lambda a, b: None)(x, y)

# Tool schema
calculator_schema = {
    "name": "calculator",
    "description": "Perform math operations",
    "parameters": {
        "operation": str,
        "x": float,
        "y": float
    }
}
```

# Building a Search Tool

---

```
import requests

def web_search(query: str, num_results: int = 3) -> str:
    """Search web and return results"""
    # Using a search API (e.g., Serper, Tavily)
    response = requests.post(
        "https://api.search.com/search",
        json={"q": query, "num": num_results},
        headers={"Authorization": f"Bearer {API_KEY}"})

    results = response.json()["results"]
    return "\n\n".join([
        f"{r['title']}\n{r['snippet']}"
        for r in results
```

# Plan-and-Execute Pattern

---

## 1. Planning Phase

| Create detailed |  
| step-by-step |  
| plan |



## 2. Execution Phase

▼  
| Execute step 1 |  
| Execute step 2 |  
| Execute step 3 |

# Python: Plan-and-Execute

---

```
def plan_and_execute(task: str, tools: dict):  
    # Step 1: Create plan  
    plan_prompt = f"Create step-by-step plan for: {task}"  
    plan = llm(plan_prompt)  
    steps = parse_steps(plan)  
  
    # Step 2: Execute each step  
    results = []  
    for step in steps:  
        result = execute_step(step, tools)  
        results.append(result)  
  
    # Step 3: Synthesize  
    synthesis = llm(f"""  
Task: {task}
```

# Memory in Agents

---

## Types of Memory

### 1. Short-term (Working memory)

- Current conversation context
- Recent observations

### 2. Long-term (Persistent)

- Past conversations
- Learned facts/patterns

# Python: Agent Memory

---

```
class AgentMemory:
    def __init__(self):
        self.short_term = [] # Recent messages
        self.long_term = {} # Vector DB
        self.facts = [] # Extracted facts

    def add_to_short_term(self, item, max_size=10):
        self.short_term.append(item)
        if len(self.short_term) > max_size:
            # Archive to long-term
            self.archive(self.short_term.pop(0))

    def archive(self, item):
        # Store in vector DB
        embedding = get_embedding(item)
```

# Reflexion Pattern

---

## Self-reflection for improvement

Attempt 1

- └─ Execute task
- └─ Evaluate result
- └─ Identify failures

Reflect

- └─ Analyze what went wrong
- └─ Generate improvement plan
- └─ Update strategy

Attempt 2

- └─ Apply lessons learned
- └─ Execute with improvements

# Python: Reflexion Agent

---

```
def reflexion_agent(task, max_attempts=3):
    reflections = []

    for attempt in range(max_attempts):
        # Execute task
        result = execute_task(task, reflections)

        # Evaluate
        success = evaluate_result(result, task)
        if success:
            return result

        # Reflect on failure
        reflection = llm(f"""
Task: {task}
```

# Multi-Agent Systems

---

## Why Multiple Agents?

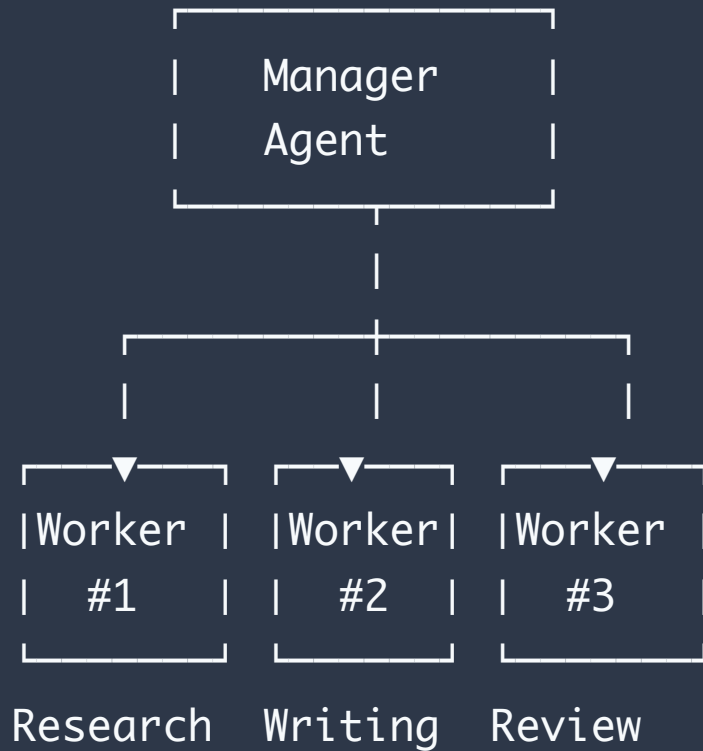
- Specialization (each agent = expert)
- Parallelization (concurrent tasks)
- Modularity (easier to debug/improve)
- Collaboration (agents work together)

## Patterns

- Hierarchical (manager + workers)
- Peer-to-peer (collaborate as equals)

# Hierarchical Multi-Agent

---



**Manager:** Delegates tasks

# Python: Multi-Agent System

---

```
class Agent:
    def __init__(self, name, role, tools):
        self.name = name
        self.role = role
        self.tools = tools

    def execute(self, task):
        prompt = f"You are {self.role}. Task: {task}"
        return llm_with_tools(prompt, self.tools)

class ManagerAgent:
    def __init__(self, workers):
        self.workers = workers

    def delegate(self, task):
```

# Agent Communication

---

## Message Passing

```
class Message:
    def __init__(self, sender, receiver, content):
        self.sender = sender
        self.receiver = receiver
        self.content = content
        self.timestamp = time.time()

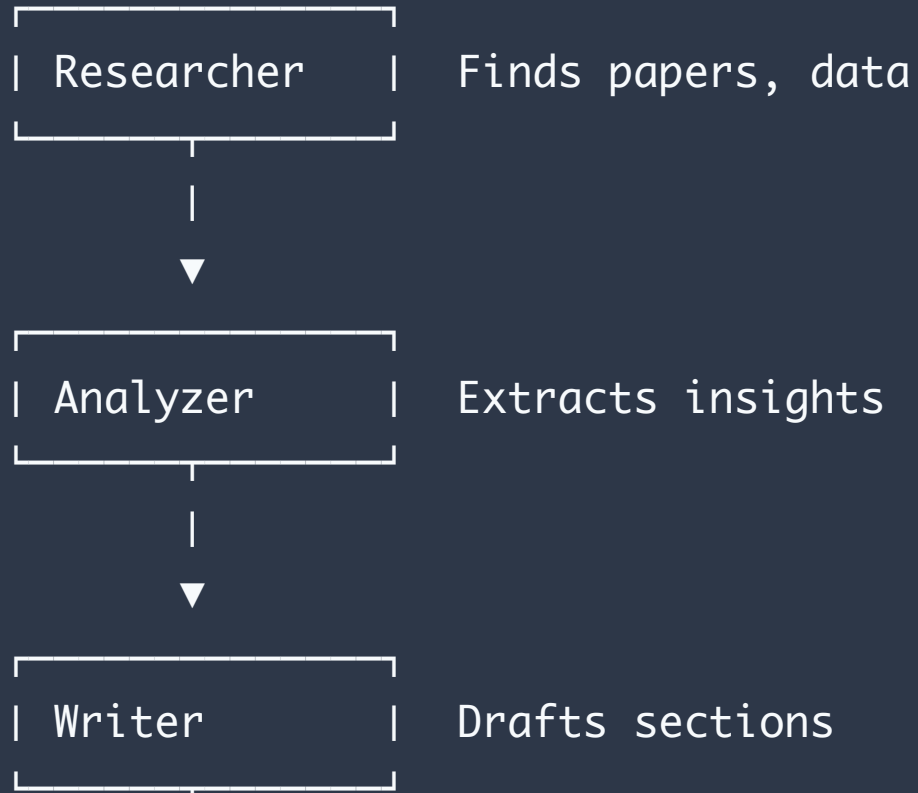
class MessageBus:
    def __init__(self):
        self.messages = []

    def send(self, msg):
        self.messages.append(msg)
```

# Agent Collaboration Example

---

## Research Paper Writer



# Tool Safety & Validation

---

## Security Concerns

- Malicious tool calls
- Data leakage
- Unintended actions
- Cost overruns

## Mitigation

```
def safe_tool_execution(tool_name, args):  
    # Validate tool exists
```

# Error Handling in Agents

---

```
def robust_agent_step(agent, task):
    max_retries = 3

    for attempt in range(max_retries):
        try:
            result = agent.execute(task)
            if validate_result(result):
                return result
            else:
                # Invalid result, retry with feedback
                task = f"{task}\nPrevious attempt invalid: {result}"

        except ToolError as e:
            logger.error(f"Tool error: {e}")
            # Try alternative tool
```

# Agent Evaluation Metrics

---

## Performance

- Task success rate
- Steps to completion
- Token usage
- Latency

## Quality

- Answer accuracy
- Tool usage appropriateness

# Python: Agent Metrics

---

```
class AgentMetrics:
    def __init__(self):
        self.steps = 0
        self.tool_calls = 0
        self.tokens = 0
        self.start_time = time.time()

    def record_step(self):
        self.steps += 1

    def record_tool_call(self, tool_name):
        self.tool_calls += 1

    def record_tokens(self, count):
        self.tokens += count
```

# LangChain Agents

---

```
from langchain.agents import AgentExecutor, create_openai_functions_agent
from langchain_openai import ChatOpenAI
from langchain.tools import Tool

# Define tools
tools = [
    Tool(
        name="Calculator",
        func=calculator,
        description="Useful for math"
    ),
    Tool(
        name="Search",
        func=web_search,
        description="Search the web"
```

# LangGraph for Complex Agents

---

```
from langgraph.graph import Graph

# Define agent workflow
workflow = Graph()

# Add nodes (agent steps)
workflow.add_node("research", research_node)
workflow.add_node("analyze", analyze_node)
workflow.add_node("write", write_node)

# Add edges (flow)
workflow.add_edge("research", "analyze")
workflow.add_edge("analyze", "write")

# Set entry point
```

# Crew AI Framework

---

```
from crewai import Agent, Task, Crew

# Define agents
researcher = Agent(
    role="Researcher",
    goal="Find relevant information",
    tools=[search_tool],
    verbose=True
)

writer = Agent(
    role="Writer",
    goal="Create engaging content",
    tools=[],
    verbose=True
```

# AutoGPT-style Agent

---

## Continuous autonomous operation

```
class AutoAgent:
    def __init__(self, goal):
        self.goal = goal
        self.memory = []
        self.tools = load_tools()

    def run(self):
        while not self.is_goal_achieved():
            # Think
            thoughts = self.think()

            # Plan
            plan = self.plan(thoughts)
```

# Agent Limitations

---

## Current Challenges

- Unreliable long-term reasoning
- Expensive (many LLM calls)
- Hard to debug
- Can go off-track
- Safety concerns

## Mitigation Strategies

- Max steps limit

# Human-in-the-Loop

---

```
class HumanApprovalAgent:
    def __init__(self):
        self.actions_requiring_approval = [
            "send_email",
            "delete_file",
            "make_payment"
        ]

    def execute_action(self, action, args):
        if action in self.actions_requiring_approval:
            print(f"Agent wants to: {action}({args})")
            approval = input("Approve? (y/n): ")
            if approval.lower() != 'y':
                return "Action rejected by user"
```

# Workshop: Research Agent

---

**Goal:** Build agent that researches topics

## Requirements

1. Search web for information
2. Extract key facts
3. Synthesize findings
4. Cite sources

## Tools Needed

# Research Agent Architecture

---

User Query

|



Query

Understanding

|

|



Search & Gather

(multiple

sources)

|

|

# Research Agent Starter

---

```
class ResearchAgent:
    def __init__(self):
        self.tools = {
            "search": web_search_tool,
            "extract": extract_content_tool
        }
        self.findings = []

    def research(self, topic: str):
        # 1. Generate search queries
        queries = self.generate_queries(topic)

        # 2. Search and collect
        for query in queries:
            results = self.tools["search"](query)
```

# Exercise 1: Basic Agent

---

**Build:** Simple ReAct agent with calculator

```
def exercise1():
    tools = {"calculator": calculator_tool}

    question = """
    If I buy 3 items at $12.50 each and 2 items
    at $8.75 each, what's my total?
    """

    agent = ReactAgent(tools)
    answer = agent.solve(question)
    print(answer)
```

# Exercise 2: Multi-Tool Agent

---

**Build:** Agent with search + calculator

```
def exercise2():
    tools = {
        "search": web_search_tool,
        "calculator": calculator_tool
    }

    question = """
    What's the GDP of the US and China combined?
    """

    agent = ReactAgent(tools)
    answer = agent.solve(question)
    print(answer)
```

# Exercise 3: Research Agent

---

**Build:** Full research agent

```
def exercise3():  
    agent = ResearchAgent(  
        tools=["search", "extract", "summarize"]  
    )  
  
    report = agent.research(  
        "What are the main applications of "  
        "transformers in NLP?"  
    )  
  
    print(report)
```

# Best Practices

---

## Agent Design

- Start simple, add complexity gradually
- Clear stopping conditions
- Limit max steps
- Comprehensive logging
- Error recovery

## Tool Design

- Clear descriptions

# Debugging Agents

---

## Common Issues

1. Infinite loops → Add max steps
2. Wrong tool selection → Improve descriptions
3. Poor reasoning → Better prompts
4. Hallucinated actions → Validate tool calls
5. Context loss → Better memory

## Debug Tools

- Verbose logging

# Production Considerations

---

## Before Deployment

- Extensive testing
- Rate limiting
- Cost monitoring
- Safety guardrails
- Rollback plan

## Monitoring

- Success/failure rates

# Advanced Agent Patterns

---

## Upcoming Topics

- RAG-enhanced agents
- Agents with persistent memory
- Multi-modal agents
- Agent-based evaluations
- Production deployment

**Day 4:** RAG + Evaluation

**Day 5:** Production systems

# Resources

---

## Frameworks

- LangChain/LangGraph
- LlamaIndex
- AutoGPT
- CrewAI
- AgentGPT

## Papers

- ReAct (Yao et al. 2022)

# Q&A

---

Questions?

**Tomorrow:** Day 4 - RAG & Evaluation

- Retrieval-Augmented Generation
- Vector databases
- Evaluation frameworks
- Testing strategies

# Thank You

---

Excellent progress!

## Homework

- Complete all 3 exercises
- Experiment with different tools
- Read ReAct paper
- Design your own agent use case

See you tomorrow!